Ejercicio: Medición del Reloj del Arduino Mega con Osciloscopio

📋 Información General

Parámetro	Valor
Microcontrolador	ATmega2560
Frecuencia del cristal	16 MHz
Período esperado	62.5 ns
Voltaje del sistema	5V

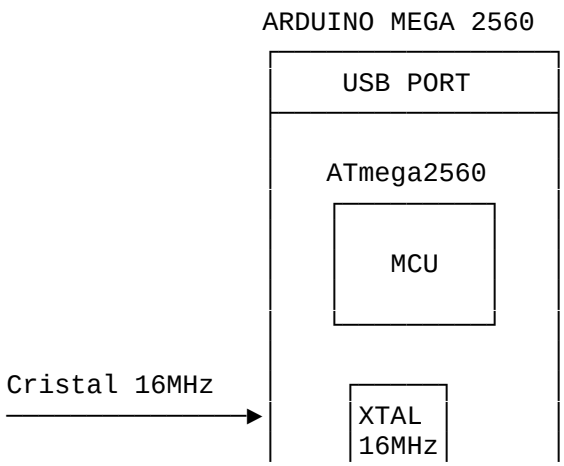
🎯 Objetivos

1. Medir la señal de reloj directamente en los pines del cristal
2. Configurar y medir la salida de reloj en un pin digital
3. Comparar ambos métodos y analizar diferencias

🛠️ Materiales Necesarios

```
- Arduino Mega 2560
- Osciloscopio (mínimo 100 MHz de ancho de banda)
- Sonda de osciloscopio (x10 recomendado)
- Cable USB para alimentar el Arduino
- Cables de conexión/jumpers
- Pinzas de precisión o puntas finas
```

📍 Ubicación de los Pines del Cristal



```
│      └──┬──┘          │
│         │             │
│     XTAL1 XTAL2       │
│     (Pin33)(Pin34)│
└───────────────────────┘
```

Vista del Cristal en la Placa

```
Cristal de 16 MHz
┌──────────────────┐
│                  │
│   Y1 16.000      │
│                  │
└──────────────────┘
   │          │
  XTAL1    XTAL2
  (Entrada)(Salida)
   │          │
  ┴┴         ┴┴
  ──         ──   ← Capacitores de carga
  ┬┬         ┬┬      (22pF típico)
  │          │
  ═══        ═══
  GND        GND
```

MÉTODO 1: Medición Directa en Pines del Cristal

⚠️ ADVERTENCIAS IMPORTANTES

```
╔═══════════════════════════════════════════════════════════╗
║  ⚡ PRECAUCIONES CRÍTICAS:                                ╒
╒                                                           ╒
╒  1. La sonda del osciloscopio AÑADE CAPACITANCIA         ╒
╒     (10-15pF típico) que puede afectar el oscilador      ╒
╒                                                           ╒
╒  2. SIEMPRE usar sonda en modo x10 para minimizar        ╒
╒     la carga capacitiva                                   ╒
╒                                                           ╒
╒  3. No aplicar presión excesiva sobre los pines          ╒
╒                                                           ╒
╒  4. Si el oscilador deja de funcionar, el Arduino        ╒
╒     se detendrá completamente                             ╒
╚═══════════════════════════════════════════════════════════╝
```

Paso 1: Preparación del Osciloscopio

```
CONFIGURACIÓN INICIAL DEL OSCILOSCOPIO
═══════════════════════════════════════
```

```
┌──────────────────────────────────────┐
│          PANEL DE CONTROL            │
├──────────────────────────────────────┤
│                                      │
│  Escala Vertical (CH1):  1V/div      │
│  Escala Horizontal:      50ns/div    │
│  Acoplamiento:           AC          │
│  Trigger:                            │
│    - Fuente: CH1                     │
│    - Modo: Auto o Normal             │
│    - Nivel: ~2.5V                    │
│    - Flanco: Subida                  │
│  Ancho de banda: Full (sin límite)   │
│  Sonda: x10                          │
│                                      │
└──────────────────────────────────────┘
```

Paso 2: Identificar los Puntos de Medición

Vista superior del cristal en Arduino Mega:

```
        ┌──────────────┐
        │  16.000 MHz  │
        │     Y1       │
        └──────────────┘
           │      │
           ●      ●
        XTAL1  XTAL2
      (entrada) (salida)
           │      │
           ▼      ▼
        Punto  Punto
          A      B
```

★ XTAL2 (Punto B) es el mejor punto de medición
porque es la salida del amplificador del oscilador

Paso 3: Conexión de la Sonda

CONEXIÓN FÍSICA

```
Sonda x10
    │
    ▼
┌─────────┐
│ Punta   │──────► Conectar a XTAL2
│         │        (pin del cristal lado derecho)
│ Clip    │
│ GND     │──────► Conectar a GND del Arduino
└─────────┘        (pin GND cercano)
```

IMPORTANTE:
- Cable de tierra lo más corto posible
- Usar el resorte de puesta a tierra si está disponible

Paso 4: Realizar la Medición

PROCEDIMIENTO:

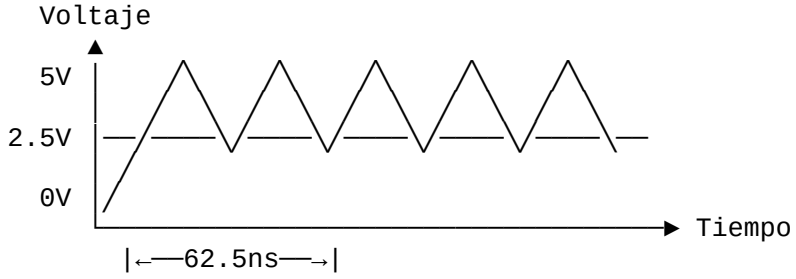
1. Encender el Arduino (conectar USB)

2. Verificar que el LED de power esté encendido

3. Tocar SUAVEMENTE con la punta de la sonda el pin XTAL2

4. Ajustar el trigger si es necesario. Presiona "Autoset" si tienes dificultades

5. Debería verse una señal similar a:



Nota: La forma de onda puede ser más sinusoidal
que cuadrada en los pines del cristal

Paso 5: Tomar Mediciones

MEDICIONES A REGISTRAR:

Parámetro	Valor Esperado	Valor Medido
Frecuencia	16.000 MHz	_____
Período	62.5 ns	_____
Voltaje pico-pico	~3-5 V	_____
Voltaje medio	~2-2.5 V	_____
Forma de onda	Sinusoidal	_____

Usar funciones automáticas del osciloscopio:
- Measure → Frequency
- Measure → Period
- Measure → Vpp

MÉTODO 2: Medición en Pin de Salida de Reloj (CLKO) (OJO!!!!) lo probaremos tras “estudiar” cómo subir el código.

Este método es **MÁS SEGURO** y **MÁS RECOMENDADO** para principiantes.

Teoría

```
El ATmega2560 puede configurar el pin PB7 para que
emita la señal de reloj del sistema (CLKOUT).

En Arduino Mega:
- PB7 corresponde al Pin Digital 13
- También tiene conectado el LED integrado

Registro involucrado: CKOUT Fuse (no modificable por software)

ALTERNATIVA: Usar Timer para generar señal derivada del reloj
```

Paso 1: Cargar el Código

```
/*
 * EJERCICIO: Salida de Reloj en Arduino Mega
 *
 * Este código configura el Timer1 para generar una señal
 * de frecuencia conocida derivada del reloj del sistema.
 *
 * Pin de salida: Pin 11 (OC1A) o Pin 12 (OC1B)
 */

void setup() {
  // ═══════════════════════════════════════════════════════
  // OPCIÓN A: Señal de 8 MHz (reloj/2) en Pin 11
  // ═══════════════════════════════════════════════════════

  // Configurar Pin 11 como salida
  pinMode(11, OUTPUT);

  // Detener Timer1
  TCCR1A = 0;
  TCCR1B = 0;

  // Modo Toggle en OC1A (Pin 11)
  // Toggle cada ciclo de reloj = f_clk / 2
  TCCR1A = (1 << COM1A0);  // Toggle OC1A on Compare Match

  // Modo CTC, sin prescaler
  TCCR1B = (1 << WGM12) | (1 << CS10);

  // Comparar en 0 para máxima frecuencia
  OCR1A = 0;
```

```
  // Frecuencia resultante: 16MHz / (2 * (1 + 0)) = 8 MHz

  Serial.begin(9600);
  Serial.println("Señal de 8 MHz en Pin 11");
  Serial.println("Conecte la sonda del osciloscopio al Pin 11");
}

void loop() {
  // El Timer genera la señal automáticamente
  // No se requiere código en el loop
}
```

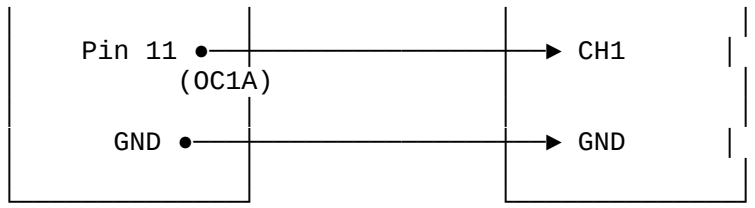
Código Alternativo: Frecuencias Variables

```
/*
 * EJERCICIO EXTENDIDO: Múltiples Frecuencias
 *
 * Genera diferentes frecuencias para practicar mediciones
 */

void setup() {
  Serial.begin(9600);
  pinMode(11, OUTPUT);  // OC1A
  pinMode(12, OUTPUT);  // OC1B

  Serial.println("=== Generador de Frecuencias ===");
  Serial.println("Comandos:");
  Serial.println("  '1' - 8 MHz (reloj/2)");
  Serial.println("  '2' - 4 MHz (reloj/4)");
  Serial.println("  '3' - 1 MHz (reloj/16)");
  Serial.println("  '4' - 500 kHz");
  Serial.println("  '5' - 100 kHz");
  Serial.println("  '6' - 1 kHz");

  // Configuración inicial: 8 MHz
  configurarFrecuencia(8000000);
}

void loop() {
  if (Serial.available()) {
    char comando = Serial.read();

    switch(comando) {
      case '1':
        configurarFrecuencia(8000000);
        Serial.println("Frecuencia: 8 MHz");
        break;
      case '2':
        configurarFrecuencia(4000000);
        Serial.println("Frecuencia: 4 MHz");
        break;
      case '3':
        configurarFrecuencia(1000000);
        Serial.println("Frecuencia: 1 MHz");
        break;
      case '4':
        configurarFrecuencia(500000);
        Serial.println("Frecuencia: 500 kHz");
        break;
      case '5':
        configurarFrecuencia(100000);
```

```
        Serial.println("Frecuencia: 100 kHz");
        break;
      case '6':
        configurarFrecuencia(1000);
        Serial.println("Frecuencia: 1 kHz");
        break;
    }
  }
}

void configurarFrecuencia(unsigned long frecuencia) {
  // Detener Timer1
  TCCR1A = 0;
  TCCR1B = 0;
  TCNT1 = 0;

  // Calcular valor de comparación
  // f_out = f_clk / (2 * prescaler * (1 + OCR1A))
  // OCR1A = (f_clk / (2 * prescaler * f_out)) - 1

  unsigned long ocr;
  byte prescaler_bits;

  if (frecuencia >= 8000000) {
    // Sin prescaler, OCR1A = 0
    prescaler_bits = (1 << CS10);
    ocr = 0;
  }
  else if (frecuencia >= 1000000) {
    // Sin prescaler
    prescaler_bits = (1 << CS10);
    ocr = (16000000UL / (2UL * frecuencia)) - 1;
  }
  else if (frecuencia >= 100000) {
    // Prescaler /8
    prescaler_bits = (1 << CS11);
    ocr = (16000000UL / (2UL * 8UL * frecuencia)) - 1;
  }
  else {
    // Prescaler /64
    prescaler_bits = (1 << CS11) | (1 << CS10);
    ocr = (16000000UL / (2UL * 64UL * frecuencia)) - 1;
  }

  // Limitar OCR1A a 16 bits
  if (ocr > 65535) ocr = 65535;

  // Configurar Timer1
  TCCR1A = (1 << COM1A0);  // Toggle OC1A
  TCCR1B = (1 << WGM12) | prescaler_bits;  // CTC mode
  OCR1A = ocr;
}
```

Paso 2: Conexión para Método 2

```
DIAGRAMA DE CONEXIÓN
═══════════════════

    ARDUINO MEGA                    OSCILOSCOPIO
    ┌─────────────┐                 ┌─────────────┐
```



Conexión más sencilla y sin riesgo para el oscilador

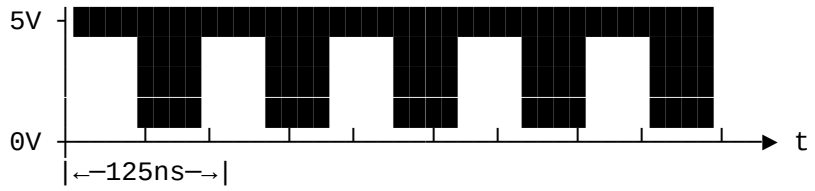
Paso 3: Configuración del Osciloscopio para Cada Frecuencia

CONFIGURACIONES RECOMENDADAS

Frecuencia	Escala Horiz	Escala Vert	Período
8 MHz	50 ns/div	2 V/div	125 ns
4 MHz	100 ns/div	2 V/div	250 ns
1 MHz	500 ns/div	2 V/div	1 µs
500 kHz	1 µs/div	2 V/div	2 µs
100 kHz	5 µs/div	2 V/div	10 µs
1 kHz	500 µs/div	2 V/div	1 ms

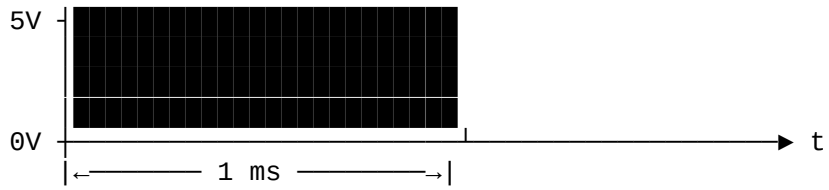
Paso 4: Señales Esperadas

SEÑAL DE 8 MHz (Pin 11)



Duty Cycle: 50%
Forma: Onda cuadrada

SEÑAL DE 1 kHz (Pin 11)



COMPARACIÓN DE AMBOS MÉTODOS

Tabla Comparativa

Característica	MÉTODO 1 (Cristal)	MÉTODO 2 (Pin 11)
Frecuencia medida	16 MHz	8 MHz (÷2)
Forma de onda	Sinusoidal	Cuadrada
Amplitud	~3-5 Vpp	0-5V (rail to rail)
Riesgo	ALTO (puede detener oscilador)	BAJO
Dificultad	ALTA	BAJA
Requiere código	NO	SÍ
Precisión	Directa	Derivada del Timer
Recomendado para	Expertos	Principiantes

Práctica: Medición de Reloj Base (16 MHz) en Arduino Mega

1. Preparación de la Sonda (Paso Crítico) (QWEN)

Antes de tocar la placa, los alumnos deben configurar la sonda:

- **Ajuste a 10x:** Mover el interruptor de la sonda a la posición **10x**.
- **Razón:** Los cristales de cuarzo son de alta impedancia. Si usan 1x, la capacitancia del cable del osciloscopio detendrá el reloj y el Arduino se "congelará" mientras midan.

2. Identificación del Cristal en el Arduino Mega

El Arduino Mega tiene dos cristales, pero el que nos interesa es el del microcontrolador principal (**ATmega2560**).

- Busquen el componente metálico alargado o el pequeño resonador cerámico (etiquetado como **Y1** o **Y2**) que dice **16.000**.
- Identifiquen los **puntos de soldadura** en la parte posterior de la placa (si es posible) o las pequeñas patas laterales si es de montaje superficial.

3. Conexión de Seguridad

Para evitar accidentes con las puntas:

1. **GND Primero:** Conecten el clip de tierra de la sonda a un pin GND del Arduino (el chasis del USB también sirve).
2. **Punto de Prueba:** Con la punta de la sonda (usando el gancho o la punta de aguja), toquen **uno de los dos extremos** del cristal.
    - *Nota:* Un pin mostrará una señal más limpia que el otro (entrada vs. salida del inversor interno). Prueben ambos.

4. Ajuste del Osciloscopio Promax

Al ser un modelo con display de frecuencia, sigan este orden para ver la onda:

1. **Voltaje (Vertical):** Ajusten a **1V/div** o **500mV/div**. La señal del cristal suele ser una onda senoidal de baja amplitud (no llega a 5V).
2. **Tiempo (Horizontal):** Ajusten a **50ns/div** o **20ns/div**.
3. **Trigger:** Si la señal "corre", giren el mando de **Trigger Level** hasta que el contador de frecuencia del display se estabilice.
4. **Acoplamiento:** Usen **AC** para centrar la onda senoidal en la pantalla.

Control	Ajuste Sugerido	Razón
VOLTS/DIV (Vertical)	1V o 2V	La señal del Arduino es de 5V. En 1V/div, ocupará 5 cuadros de alto.
TIME/DIV (Horizontal)	0.1 µs o 0.05 µs	A 8 MHz, un ciclo dura 125 ns. Necesitamos una escala muy rápida.
POSITION (V y H)	Centrado	Para que la señal no quede "escondida" fuera de la pantalla.

5. Lectura del Display de Frecuencia

Una vez que la onda está estable en la pantalla:

- Observen el **display digital** del Promax.
- Debería marcar algo muy cercano a **16.000 MHz** (o 16.000.000 Hz).
- **Análisis:** Si marca 15.998 o 16.002, explícales que es normal debido a la tolerancia del cristal y la temperatura.

6. Advertencia para los Alumnos (El "Efecto Carga")

Si al tocar el cristal el Arduino deja de parpadear (si tenía un LED de prueba) o la frecuencia cae drásticamente:

- Significa que la sonda está afectando al circuito.
- **Solución:** Asegurarse de que el osciloscopio esté configurado para reconocer que la sonda está en **10x** para que la lectura de voltaje sea real, aunque la frecuencia no debería verse afectada por esto.